

Modern Java Interview Roadmap → Spring Boot

2026 edition • Java 17/21/25 mindset • Interview-focused • Obsolete/rare legacy topics intentionally excluded

How to read the stars

★★★ Deep: learn thoroughly, code it, and be ready for interview questions. ★★ Medium: understand well and practice a few examples. ★ Surface: know what it is, why it exists, and basic usage; don't spend much time initially.

Important: Stars indicate practical interview depth, not official Java importance. Exact interview emphasis varies by company and role.

The shortest answer: when can I start Spring Boot?

Stage	What you should know	Status
1	Java syntax, types, control flow, methods	★★★
2	Classes, objects, constructors, OOP, interfaces, inheritance, polymorphism	★★★
3	String, arrays, enums, records basics	★★★
4	Exceptions + try/catch/try-with-resources	★★★
5	Collections: List/Set/Map/Queue + HashMap internals	★★★
6	Generics + Comparable/Comparator	★★★
7	Java 8: lambda, functional interfaces, streams, Optional	★★★
8	Modern Java: var, switch expressions, records, pattern matching, sealed classes	★★
9	Maven/Gradle basics + packages/imports	★★

Once you complete stages 1–7, you are ready to start Spring Boot. Do not wait until you finish JVM internals, advanced concurrency, reflection, networking, or every modern Java feature. Spring itself will teach you additional concepts as you use it.

This aligns with current Spring: Spring Boot 4.1 requires Java 17+, and Spring's core is built around IoC/DI, beans, configuration and component scanning. Maven or Gradle is recommended for dependency management.

PART A — Core Java you actually need

1. Java setup & execution

- ★★★ JDK vs JVM vs bytecode; compile/run flow (javac, java)
- ★★ JDK/JRE terminology and what the JVM actually does
- ★★ Classpath and packages/imports
- ★ IDE setup, JShell
- ★ Java release/LTS concept — know 17, 21, 25 and why projects choose an LTS

2. Language fundamentals

- ★★★ Primitive types vs reference types
- ★★★ Variables, operators, expressions
- ★★★ if/else, switch, loops, break/continue
- ★★★ Methods, parameters, return values, pass-by-value
- ★★★ Scope, static, final
- ★★ Type casting, widening/narrowing
- ★★ var (Java 10): local-variable type inference
- ★ Text blocks (Java 15)

3. OOP — very important

- ★★★ Class, object, fields, methods, constructors
- ★★★ Encapsulation
- ★★★ Inheritance and composition
- ★★★ Method overloading vs overriding
- ★★★ Runtime polymorphism / dynamic dispatch
- ★★★ Abstract classes
- ★★★ Interfaces and default methods
- ★★★ Access modifiers
- ★★★ this and super
- ★★ static vs instance members
- ★★ final class/method/variable
- ★★ Composition vs inheritance — when to prefer each

4. Object, String & common types

- ★★★★ Object: equals(), hashCode(), toString()
- ★★★★ String immutability
- ★★★★ String pool / interning concept
- ★★ StringBuilder vs StringBuffer
- ★★ Wrapper classes and autoboxing/unboxing
- ★★ Enums
- ★★ BigDecimal for money/precision
- ★ Character/regex APIs — only practical usage initially

5. Arrays & data handling

- ★★★★ 1D/2D arrays, iteration, common operations
- ★★ Arrays utility methods
- ★★ Varargs
- ★ Primitive/reference-array subtleties beyond interview basics

6. Exceptions

- ★★★★ Checked vs unchecked exceptions
- ★★★★ try/catch/finally
- ★★★★ throw vs throws
- ★★★★ Custom exceptions
- ★★★★ Exception propagation and stack traces
- ★★ try-with-resources / AutoCloseable
- ★ Exception design conventions in large applications

7. Collections — very important

- ★★★★ Collection hierarchy and when to use List/Set/Map/Queue/Deque
- ★★★★ ArrayList vs LinkedList
- ★★★★ HashSet vs TreeSet vs LinkedHashSet
- ★★★★ HashMap vs TreeMap vs LinkedHashMap
- ★★★★ HashMap internals: hashing, buckets, collisions, resizing, load factor
- ★★★★ equals() + hashCode() relationship in hash collections
- ★★★★ Iterator and enhanced for-loop
- ★★★★ Comparable vs Comparator
- ★★ PriorityQueue
- ★★ Deque / ArrayDeque
- ★★ Immutable/unmodifiable collections and factory methods (List.of, etc.)
- ★ SequencedCollection (Java 21)
- ★ Legacy Vector, Hashtable, Stack — recognize them; don't study deeply

8. Generics — very important

- ★★★★ Why generics exist
- ★★★★ Generic classes/methods
- ★★★★ Type parameters and bounded types
- ★★★★ Wildcards: ?, extends, super
- ★★★★ PECS
- ★★ Type erasure
- ★★ Generic collections in real code

9. Java 8 functional programming — essential

- ★★★★ Lambda expressions
- ★★★★ Functional interfaces
- ★★★★ Predicate, Function, Consumer, Supplier
- ★★★★ Method references
- ★★★★ Stream pipeline: source → intermediate → terminal
- ★★★★ map/filter/flatMap
- ★★★★ sorted/distinct/limit/skip
- ★★★★ collect, groupingBy, partitioningBy
- ★★★★ reduce
- ★★ Optional: map/orElse/orElseGet/orElseThrow
- ★★ Parallel streams — know risks; don't overuse
- ★★ Default/static interface methods

10. Date/time

- ★★ java.time: LocalDate, LocalDateTime, Instant, ZonedDateTime
- ★★ Duration vs Period
- ★★ Formatting/parsing with DateTimeFormatter
- ★ Know that old Date/Calendar APIs exist for legacy code; don't learn them as your primary API

PART B — Modern Java (learn after the Java 8 foundation)

- ★★★ Records (Java 16): concise immutable data carriers; understand canonical constructor, accessors, equality
- ★★ Pattern matching for instanceof (Java 16)
- ★★ Switch expressions (Java 14)
- ★★ Sealed classes/interfaces (Java 17)
- ★★ Pattern matching for switch + record patterns (Java 21)
- ★★ Text blocks (Java 15)
- ★★ var (Java 10)
- ★ Sequenced collections (Java 21)
- ★ Unnamed variables/patterns (Java 22)
- ★ Java 25 additions: module import declarations, compact source files/instance main methods, flexible constructor bodies
- ★ Preview/incubator features: know that they exist; do not make them a study priority unless your project uses them

Version rule: you do NOT need to memorize every Java release. Learn the concept and remember the milestone version for the major modern features. Oracle's language-change documentation lists the permanent milestones: switch expressions (14), records/pattern matching for instanceof (16), sealed classes (17), and record patterns/pattern matching for switch (21).

PART C — Concurrency & JVM: important, but after Spring starts

- ★★★ Thread basics, Runnable, Callable
- ★★★ Race conditions, thread safety, synchronization
- ★★★ synchronized
- ★★★ ExecutorService / thread pools
- ★★★ Future and CompletableFuture basics
- ★★★ Virtual threads (Java 21): what problem they solve and when they help
- ★★ volatile and visibility
- ★★ Atomic classes
- ★★ Lock / ReentrantLock
- ★★ ConcurrentHashMap and concurrent collections
- ★★ Java Memory Model: happens-before at interview level
- ★ ForkJoinPool / work-stealing
- ★ CountdownLatch, CyclicBarrier, Semaphore — recognize and use when needed
- ★ Structured concurrency / scoped values / preview features — surface until actually required

JVM & memory

- ★★★ Stack vs heap vs metaspace
- ★★★ Object allocation and references
- ★★★ Garbage collection fundamentals and GC roots

★★ G1 GC at a conceptual level

★★ JIT compilation

★★ Class loading basics

★ JFR/JMC, profiling and troubleshooting — learn when doing production/performance work

★ ZGC/Shenandoah implementation details — surface initially

PART D — Build tools & testing before/alongside Spring

- ★★★ Maven: pom.xml, dependencies, lifecycle, plugins, profiles basics
- ★★ Gradle: understand the role and basic syntax; Maven is enough as your first build tool
- ★★★ JUnit 5: test structure, assertions, lifecycle, parameterized tests basics
- ★★ Mockito: mock, when/thenReturn, verify, argument matchers
- ★★ Testcontainers — useful for integration tests with real DB/brokers
- ★ JaCoCo / coverage concepts
- ★ CI test execution basics

PART E — What NOT to spend time on initially

- Old Java language syntax/features simply because a tutorial teaches them historically.
- Applet development, Java Web Start, SecurityManager-era application design, RMI/JAX-RPC-era enterprise material.
- Deep XML-based Spring configuration when learning modern Spring Boot.
- Legacy collection classes such as Vector/Hashtable/Stack beyond recognition.
- Old date APIs as your primary approach (Date/Calendar).
- Deep Java module-system details unless your project is modular.
- Reflection internals and MethodHandles beyond a basic understanding of what frameworks use.
- Every preview/incubator feature from Java 18–25.
- Deep GC algorithm implementation details before you have production/performance needs.
- Trying to memorize every Java version number. Focus on modern milestones.

PART F — Spring Boot entry point

- ★★★ Spring IoC / Dependency Injection mental model
- ★★★ Bean, ApplicationContext, component scanning
- ★★★ Constructor injection
- ★★★ @Component, @Service, @Repository, @Controller
- ★★★ @Configuration and @Bean
- ★★ Bean scopes and lifecycle
- ★★ @Qualifier / multiple implementations
- ★★ Configuration/properties basics
- ★★ Spring Boot auto-configuration concept
- ★★ Starter dependencies
- ★★ REST controller basics

Spring's current documentation describes IoC/DI as the core model and shows Java configuration through `@Configuration` and `@Bean`, while component scanning discovers stereotypes such as `@Component`, `@Service`, `@Repository`, and `@Controller`.

Recommended sequence for you

1. Java fundamentals + OOP
2. String/arrays + exceptions
3. Collections + HashMap internals
4. Generics
5. Java 8 lambdas + functional interfaces + Streams + Optional
6. Maven basics
7. Start Spring Core / Spring Boot immediately
8. Learn records, switch expressions, pattern matching, sealed classes while building Spring
9. Then concurrency + JVM + GC for stronger backend interviews
10. Build one Spring Boot REST service and use PostgreSQL, JPA/Hibernate, validation, exception handling, tests, security, and Docker

Interview depth map

Topic	Depth	Typical reason
OOP	★★★	Core Java + Spring design understanding
Collections	★★★	Daily Java + common interview questions
HashMap	★★★	Frequently tested internals
Generics	★★★	APIs, collections, framework code
Java 8	★★★	Modern Java codebase fundamentals
Exceptions	★★★	Production/backend code

Streams	★★★	Modern Java coding/questions
Concurrency	★★★	Backend/SDE interviews, production
Virtual threads	★★★	Common 'what changed in modern Java' question
JVM/GC	★★	Backend interviews; deepen with experience
Records/pattern matching	★★	Modern Java literacy
Maven	★★★	Spring project/build workflow
Reflection/modules/advanced JVM	★	Know concept; specialize later

Final target

Do not aim to “finish Java.” Aim to become comfortable writing Java backend code. For your Spring transition, the high-value core is: OOP → Collections → Generics → Exceptions → Java 8 functional programming/Streams → Maven. Then start Spring Boot. Concurrency, JVM, GC and advanced Java become the second layer for stronger SDE/backend interviews.

Research basis: Oracle Java language/API documentation and the official Spring Framework/Spring Boot documentation, checked for the current 2026 documentation baseline. Spring Boot 4.1 currently requires Java 17 or newer and recommends Maven or Gradle for dependency management.